
Learning Permutation Distributions via Reflected Diffusion on Ranks

Sizhuang He^{*1} Yangtian Zhang^{*1} Shiyang Zhang¹ David van Dijk¹

Abstract

The finite symmetric group S_N provides a natural domain for permutations, yet learning probability distributions on S_N is challenging due to its factorially growing size and discrete, non-Euclidean structure. Recent permutation diffusion methods define forward noising via shuffle-based random walks (e.g., riffle shuffles) and learn reverse transitions with Plackett–Luce (PL) variants, but the resulting trajectories can be abrupt and increasingly hard to denoise as N grows. We propose *Soft-Rank Diffusion*, a discrete diffusion framework that replaces shuffle-based corruption with a structured soft-rank forward process: we lift permutations to a continuous latent representation of order by relaxing discrete ranks into soft ranks, yielding smoother and more tractable trajectories. For the reverse process, we introduce *contextualized generalized Plackett–Luce (cGPL)* denoisers that generalize prior PL-style parameterizations and improve expressivity for sequential decision structures. Experiments on sorting and combinatorial optimization benchmarks show that Soft-Rank Diffusion consistently outperforms prior diffusion baselines, with particularly strong gains in long-sequence and intrinsically sequential settings.

1. Introduction

Permutations are fundamental objects in a wide range of applications, from ranking search results and recommendations (Feng et al., 2021; Koren et al., 2009) to sorting and learning-to-rank (Liu, 2009) and combinatorial optimization problems such as the traveling salesperson problem (TSP). In many settings, we aim to model a *distribution* over permutations—and ultimately build generative models with which to sample permutations.

^{*}Equal contribution ¹Department of Computer Science, Yale University, New Haven, CT, USA. Correspondence to: David van Dijk <david.vandijk@yale.edu>, Sizhuang He <sizhuang.he@yale.edu>.

In continuous Euclidean domains, diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021; 2022) have emerged as powerful and scalable tools for modeling complex distributions like images, and related ideas have been extended beyond $\mathbb{R}^d$ to discrete objects such as text and graphs, a class of models known as Discrete Diffusion models (Austin et al., 2023; Lou et al., 2024; Dieleman et al., 2022; Gat et al., 2024; Gulrajani and Hashimoto, 2023; Vignac et al., 2023). Yet permutations remain a particularly challenging case: although they are discrete, the state space grows factorially with sequence length N , and the natural transitions on permutations, like card-shuffling, are often *abrupt*—small local moves can cause discontinuous, non-differentiable changes in the induced ordering. Consequently, diffusion-style constructions that work well for other discrete domains can become brittle on permutations and scale poorly with N .

Developing discrete diffusion models over permutations remains relatively underexplored. Recent progress on permutation diffusion (Zhang et al., 2025) tackles above challenge by defining the forward noising process directly on permutations—typically as a random walk induced by *riffle shuffles* (Gilbert, 1955)—and parameterizing the reverse dynamics with the Plackett–Luce distribution (PL) (Plackett, 1975; Luce, 1959) and its generalizations (GPL) (Zhang et al., 2025). While effective on small instances, these discrete forward trajectories can be highly “jumpy”: even a single riffle–shuffle can simultaneously move many items across the sequence, yielding abrupt, non-smooth changes in the ordering. Performance often degrades rapidly and can even collapse in the long- N regime.

In this paper, we take a different perspective: rather than diffusing *within* S_N , we lift a permutation—viewed as the rank of each element—to a continuous latent representation and diffuse in that space. Concretely, we represent an ordering by assigning each item a continuous-valued *soft rank* $z \in [0, 1]$, obtaining a soft rank vector $Z \in [0, 1]^N$ and recover the induced permutation by sorting, $\sigma = \text{argsort}(Z)$. This relaxation lets us define smooth stochastic dynamics in a continuous space, while still producing a discrete permutation at any time via a simple sorting operation.

Building on this idea, we introduce **Soft-Rank Diffusion**, which defines the forward noising process as a reflected

diffusion bridge (Lou and Ermon, 2023; Xie et al., 2024) on $[0, 1]^N$, and observes permutations through the induced ordering $\sigma_t = \text{argsort}(Z_t)$. The latent construction also yields a reverse-time sampler. We augment the *intractable* discrete reverse step with an auxiliary, *tractable* continuous update in the soft-rank space, followed by a projection back to permutations via sorting. We further introduce **contextualized GPL (cGPL)** and a **pointer-cGPL** variant, *generalizations* of PL/GPL that make stagewise logits depend on the evolving prefix and the shrinking candidate set—a natural fit for intrinsically sequential and dynamic tasks such as TSP.

We evaluate Soft-Rank Diffusion on standard permutation generation benchmarks, including 4-digit MNIST sorting, TSP, and CIFAR-10 (Krizhevsky, 2009) jigsaw puzzles. Across settings, our method consistently outperforms prior permutation diffusion baselines and differentiable sorting baselines, with gains that widen as sequence lengths increase. Taken together, our results suggest that reflected diffusion in soft-rank space provides a principled and scalable route to permutation generative modeling.

Contributions. Our main contributions are:

- We introduce **Soft-Rank Diffusion**, a permutation diffusion framework induced by reflected diffusion bridges in a relaxed continuous soft-rank space.
- We derive a **hybrid reverse sampler** that augments the intractable reverse dynamics in permutation space with tractable updates in the soft-rank space.
- We propose **cGPL** and **pointer-cGPL**, generalizations of PL/GPL that improve prefix-conditional expressivity for sequential permutation tasks.
- We demonstrate strong empirical performance on long-sequence MNIST sorting and TSP benchmarks, particularly in the large- N regime.

2. Related Works

Discrete Diffusion Models. Diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021) were initially developed for continuous data. D3PM (Austin et al., 2023) extended this framework to discrete domains by defining forward noising processes based on masking or uniform replacement. Subsequent work introduced alternative parameterizations and training objectives for discrete diffusion (Lou et al., 2024; Gat et al., 2024) and further extended discrete (both in state space and time) diffusion to continuous-time variants (Campbell et al., 2022; Sun et al., 2023; Shi et al., 2025). However, most existing methods assume a manageable or factorized state space; directly extending them to permutation-valued data is challenging

because the group S_N has size $N!$, making naive transition representations intractable without exploiting additional structure.

Learning Permutations. Sorting algorithms can be viewed as producing a permutation of a set of items and are well understood; however, their hard discrete decisions are *non-differentiable*, which hinders end-to-end training when permutations are produced by, or embedded within, neural models. This motivates differentiable sorting methods that replace discrete swaps or permutation operators with smooth surrogates, yielding a *soft* permutation (or rank) representation amenable to gradient-based optimization (Mena et al., 2018; Cuturi et al., 2019; Prillo and Eisenschlos, 2020; Blondel et al., 2020; Grover et al., 2019; Petersen et al., 2022).

Another widely used family is **Pointer Networks** (Ptr-Nets) (Vinyals et al., 2017), which parameterize permutation-valued outputs by decoding a sequence of indices into the input, rather than symbols from a fixed vocabulary. At each decoding step, an attention mechanism induces a categorical distribution over input positions, and the chosen index specifies which input element is appended next in the output ordering. Because the effective output domain scales with input length, Ptr-Nets naturally accommodate variable-sized instances and provide a natural generic parameterization for combinatorial prediction problems whose outputs are orderings (e.g., sorting and routing).

Discrete Diffusion on Permutations. Zhang et al. (2025) introduce *SymmetricDiffusers*, which formulates discrete diffusion directly on the finite symmetric group S_N . The forward noising dynamics are instantiated as a random walk on S_N , with the *riffle shuffle* (Gilbert, 1955) serving as an effective transition that mixes rapidly to enable short diffusion chains in practice.

For the reverse (denoising) parameterization, Plackett–Luce-style (PL) models are adopted and extended to a generalized Plackett–Luce (GPL) family. While the PL distribution is defined as

$$p_{\text{PL}}(\sigma) = \prod_{i=1}^N \frac{\exp(s_{\sigma(i)})}{\sum_{j=i}^N \exp(s_{\sigma(j)}), \quad (1)$$

i.e., it assigns a single preference score s_k to each item k and samples items *without replacement* according to these fixed scores, the GPL family generalizes this construction by assigning step-specific scores for each item rather than a single fixed global score. As a result, GPL is strictly more expressive than PL; in particular, GPL is universal in the sense that it can represent arbitrary distributions over S_N .

Reflected Diffusion and Reflected Flow Matching. Diffusion models are often trained on data supported on

bounded domains (e.g., $[0, 255]$ for unnormalized image pixels), yet the learned dynamics can leave the domain at intermediate time steps—a pathology that is commonly handled by ad-hoc clipping. Lou and Ermon (2023) address this issue by formulating generation as a *reflected* score-based SDE, augmenting the dynamics with an additional reflection term that enforces the state constraint and keeps trajectories in-bounds throughout sampling. Xie et al. (2024) later extend the same principle to flow matching, incorporating reflection into the corresponding flow/ODE formulations.

3. Methods

Let $[N] := \{1, \dots, N\}$. A permutation is a bijection $\sigma : [N] \rightarrow [N]$. We can write it as

$$\sigma = \begin{pmatrix} 1 & 2 & \cdots & N \\ \sigma(1) & \sigma(2) & \cdots & \sigma(N) \end{pmatrix},$$

meaning that each element $i \in [N]$ is mapped to $\sigma(i)$. We can make the source indices $(1, 2, \dots, N)$ implicit and use the one-line notation

$$\sigma = (\sigma(1), \sigma(2), \dots, \sigma(N)).$$

Under this convention, $\sigma(i)$ can be interpreted as the (destination) position of element i after reordering, i.e., the **rank** of element i of the original sequence in the permuted ordering.

We denote by $\mathcal{S}_N$ the set of all such permutations (i.e., the symmetric group under composition). Applying a permutation to an ordered list simply reindexes its entries. Equivalently, each $\sigma \in \mathcal{S}_N$ can be represented by a permutation matrix $P_\sigma \in \{0, 1\}^{N \times N}$, so that the same reindexing can be implemented via matrix multiplication.

Given an instance $X = (x_1, \dots, x_N) \in \mathbb{R}^{N \times d}$ consisting of N items (with associated features), we aim to model a conditional distribution over permutations $\sigma \in \mathcal{S}_N$. We observe i.i.d. training pairs $(X, \sigma_0) \sim p^*$, and our goal is to learn a generative model that can efficiently sample valid permutations conditioned on X . We write $X_t = \sigma_t(X)$ to be the permuted instance at time t .

We adopt a diffusion-style approach on $\mathcal{S}_N$. Specifically, we define a forward noising process on permutations that gradually transforms σ_0 toward a target distribution, and train a parameterized reverse-time model $p_\theta(\sigma_{t-\Delta t} \mid \sigma_t, X)$ to approximately invert this corruption process. At test time, sampling starts from the reference distribution and iteratively applies the learned reverse dynamics to generate $\hat{\sigma}_0 \in \mathcal{S}_N$ conditioned on X .

3.1. Forward Process: Soft-Rank Diffusion

Instead of defining diffusion dynamics directly on the discrete space $\mathcal{S}_N$ (e.g., via card shuffling methods as in Zhang

et al. (2025), which can be abrupt and unstructured), we adopt a continuous latent viewpoint in which a permutation is encoded by the relative ordering of real-valued coordinates.

The key idea is to relax the discrete rank values in $[N]$ to continuous *soft ranks* in $[0, 1]$, and to define a diffusion process over these continuous variables. At any intermediate time, the induced ordering of the latent coordinates yields a valid permutation of the original items.

This relaxation brings two immediate benefits. First, it provides a Euclidean state space where noise injection and interpolation are natural, enabling diffusion-style modeling without relying on discrete, jump-like transitions. Second, the forward marginals can be designed to be analytically tractable, which later allows us to derive a reverse-time sampler.

Intuitively, one may view each item as a particle undergoing Brownian motion: starting from an initial ordered configuration, the particles gradually diffuse, and the permutation at time t is given by the snapshot ordering of their positions.

Formally, to initialize a continuous latent state, we convert a permutation $\sigma \in \mathcal{S}_N$ into a canonical *soft-rank* vector in $[0, 1]^N$ by mapping discrete ranks to a fixed grid. Let $\{g_r\}_{r=1}^N$ be a uniform grid on $[0, 1]$, e.g.,

$$g_r = \frac{r-1}{N-1} \quad (r = 1, \dots, N). \quad (2)$$

We define the grid-mapping operator $\text{LiftToGrid} : \mathcal{S}_N \rightarrow [0, 1]^N$ by

$$\text{LiftToGrid}(\sigma)_i := g_{\sigma(i)}, \quad i \in [N]. \quad (3)$$

In particular, we set $Z_0 := \text{LiftToGrid}(\sigma_0)$.

We generate Z_t by coupling the data latent Z_0 to a randomly sampled endpoint $Z_1 \sim p_{\text{ref}}$ via a Brownian bridge, where p_{ref} is a tractable, data-independent reference distribution (e.g., $\mathcal{N}(0, I_N)$ or $\text{Unif}([0, 1]^N)$). We apply this construction *coordinate-wise*: each entry of Z_t evolves as an independent one-dimensional bridge driven by its own Brownian motion. For clarity, we describe a single coordinate using lowercase notation. Specifically, we consider the following SDE, which corresponds to a VE diffusion bridge as in Zhou et al. (2023):

$$dz_t = \frac{z_1 - z_t}{1-t} dt + \eta dw_t + dl_t, \quad (4)$$

where w_t is a standard one-dimensional Brownian motion, η is the noise scale and l_t is a reflection term (Lou and Ermon, 2023; Xie et al., 2024) that enforces the state constraint $z_t \in [0, 1]$.

Then, at any time t , we obtain a discrete permutation in the rank representation simply by ranking the coordinates

of Z_t . Assuming distinct coordinates (almost surely under continuous noise) and using 1-based indexing, we recover σ_t as

$$\sigma_t = \text{argsort}(\text{argsort}(Z_t)).$$

Figure 1 illustrates the forward process.

3.2. Reverse Process

Although the forward process is defined in a continuous latent space, our observations and learning targets are the induced permutations. In other words, the model operates on discrete permutation-valued states, while the continuous latent dynamics serve as an underlying construction that we invoke only to derive a convenient reverse-time update. One can choose to treat the continuous latent Z_t as the primary reverse-time state and learn a denoiser directly in $\mathbb{R}^N$. We instead work in $\mathcal{S}_N$: during sampling we keep the permutation σ_t as the explicit state, and reconstruct a rank-consistent latent Z_t only as an auxiliary variable when executing each backward update.

We adopt a σ_0 -prediction style parameterization. Specifically, a neural network f_θ predicts an estimate of the initial permutation,

$$\hat{\sigma}_0 = f_\theta(X_t, t), \quad (5)$$

which in turn induces an estimate of the clean sequence via the permutation action, $\hat{X}_0 = \hat{\sigma}_0(X)$.

We describe the overall reverse process next, and later provide details on the parameterization of f_θ .

Reverse-time transition kernel. Our forward process in Eq. 4 is a *reflected* Brownian bridge. Following Xie et al. (2024), in practice one first solve an *unconstrained* backward step in $\mathbb{R}^N$ using the corresponding *unreflected* bridge conditional, and apply the reflection operator only afterwards (and only when needed).

Specifically, let $\mu(u) := (1 - u)z_0 + uz_1$ denote the linear interpolation between endpoints. For the *unconstrained* bridge, for any $0 < s < t < 1$ we have the closed-form Gaussian conditional

$$z_s \mid z_t, z_0, z_1 \sim \mathcal{N}\left(\mu(s) + \frac{s}{t}(z_t - \mu(t)), \eta^2 \frac{s(t-s)}{t}\right). \quad (6)$$

Equivalently, we sample a proposal $\tilde{z}_s$ by

$$\tilde{z}_s = \mu(s) + \frac{s}{t}(z_t - \mu(t)) + \eta \sqrt{\frac{s(t-s)}{t}} \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1). \quad (7)$$

and then enforce the latent-domain constraint via reflection:

$$z_s \leftarrow \mathcal{R}(\tilde{z}_s). \quad (8)$$

Algorithm 1 Reverse Sampling via Reflected Gaussian-Bridge Updates

Require: Number of items n ; time grid $0 = t_0 < t_1 < \dots < t_K = 1$; diffusion scale η .
 Reference distribution p_{ref} on $[0, 1]^n$.
 Score network f_θ defining $p_\theta(\sigma \mid X_t, t)$.
 Operators $\text{LiftToGrid}(\cdot)$ and $\text{Reflect}(\cdot)$.

- 1: Sample $z_1 \sim p_{\text{ref}}$ and set $z_{t_K} \leftarrow z_1$.
- 2: **for** $k = K, K-1, \dots, 1$ **do**
- 3: $t \leftarrow t_k, \quad s \leftarrow t_{k-1}$.
- 4: $X_t \leftarrow \text{argsort}(z_t)$ (induce discrete state)
- 5: Sample $\hat{\sigma}_0 \sim p_\theta(\cdot \mid X_t, t)$ (via f_θ)
- 6: $\hat{z}_0 \leftarrow \text{LiftToGrid}(\hat{\sigma}_0) \in [0, 1]^n$.
- 7: $\mu_t \leftarrow (1-t)\hat{z}_0 + tz_1, \quad \mu_s \leftarrow (1-s)\hat{z}_0 + sz_1$.
- 8: Sample $\xi \sim \mathcal{N}(0, I_n)$.
- 9: $\tilde{z}_s \leftarrow \mu_s + \frac{s}{t}(z_t - \mu_t) + \eta \sqrt{\frac{s(t-s)}{t}} \xi$.
- 10: $z_s \leftarrow \text{Reflect}(\tilde{z}_s)$.
- 11: $z_t \leftarrow z_s$.
- 12: **end for**
- 13: **return** z_{t_0} and optionally $\hat{\sigma} \leftarrow \text{argsort}(z_{t_0})$.

We defer the derivation of Eq. (6) to Appendix B.

The reverse sampling procedure is initialized by drawing $z_1 \sim p_{\text{ref}}$. Since our model maintains only a permutation-valued state, we obtain the corresponding initial permutation by mapping the latent to the grid via $\sigma_1 = \text{LiftToGrid}(z_1)$ in Eq. (3). The complete reverse-time algorithm is summarized in Algorithm 1 and illustrated in Figure 1.

3.3. Model architecture

We now describe the model architecture used to parameterize the σ_0 -prediction distribution and to sample an estimate of the clean permutation, as in Eq. 5.

We extend the *Generalized Plackett–Luce* (GPL) parameterization of (Zhang et al., 2025) to a *contextualized Generalized Plackett–Luce* (cGPL) model. In contrast to the encoder-only architecture used in Zhang et al. (2025), we implement cGPL with a full encoder–decoder Transformer (Vaswani et al., 2023). This formulation generalizes GPL, and we show empirically that cGPL yields a more expressive parameterization across our benchmarks.

Contextualized GPL. Similar to GPL (Zhang et al., 2025), cGPL models a permutation $\sigma \in \mathcal{S}_N$ with a stagewise distribution that, at each position i , normalizes scores over the remaining (unselected) set $\mathcal{R}_i(\sigma) := [N] \setminus \{\sigma(1), \dots, \sigma(i-1)\}$:

$$p_\theta(\sigma \mid X, t) = \prod_{i=1}^N \frac{\exp(s_{\sigma(i), i})}{\sum_{k \in \mathcal{R}_i(\sigma)} \exp(s_{k, i})}. \quad (9)$$

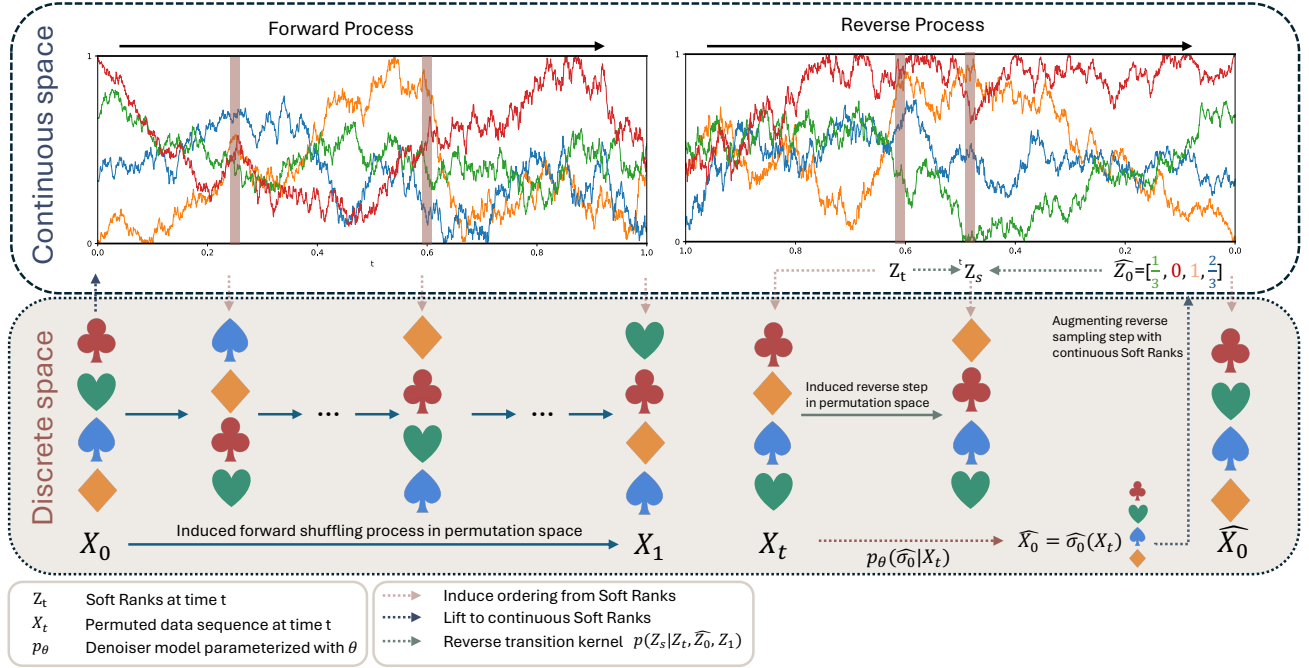


Figure 1. Soft-Rank Diffusion. We define the forward diffusion by relaxing each item’s rank to a continuous *soft-rank* variable and evolving these soft ranks in a reflected diffusion process (Lou and Ermon, 2023). At each time t , the soft ranks induce a discrete ordering by simple sorting, thereby yielding a forward process in permutation space. For reverse sampling, we couple a discrete denoiser in permutation space (predicting a clean permutation from X_t) with an auxiliary continuous update: we lift the predicted permutation to a grid-aligned soft-rank vector $\hat{Z}_0$, sample an intermediate latent Z_s for $s < t$ from a conditional reverse kernel $p(Z_s | Z_t, \hat{Z}_0, Z_1)$, and map back to permutation space by sorting Z_s , thus stepping backward in time.

The distinction between GPL and cGPL lies entirely in the *context* each position- i score vector $s_{:,i}$ is allowed to depend on. In GPL, scores are prefix-agnostic and can be computed once as a static matrix. Consequently, sampling from GPL consists of a single forward pass to obtain all scores, followed by sequentially reading the corresponding columns and sampling without replacement.

GPL:

$$s_{k,i} := [f_\theta(X, t)]_{k,i}, \quad (10)$$

In contrast, cGPL makes scores explicitly prefix-conditional through an encoder-decoder Transformer. As a result, sampling from cGPL must proceed autoregressively: scores are recomputed progressively as the prefix $\sigma_{<i}$ is instantiated.

cGPL:

$$s_{k,i} = [f_\theta(X, \sigma_{<i}, t)]_{k,i}. \quad (11)$$

See Algorithm 2 for the complete sampling procedure of cGPL. Empirically, prefix-conditioning improves performance across all tasks we consider, and the benefits are particularly pronounced on intrinsically sequential and dynamic settings such as the Traveling Salesperson Problem, where next-step decisions depend strongly on the evolving partial solution.

When the decoder is made prefix-agnostic such that

$s_{k,i}(X, \sigma_{<i}, t) \equiv s_{k,i}(X, t)$ for all prefixes, cGPL reduces to GPL; hence we can conclude cGPL generalizes GPL.

Figure 2 demonstrates the difference among sampling from PL, GPL and cGPL.

In practice, both sampling and likelihood evaluation require normalizing scores only over the remaining (feasible) set $\mathcal{R}_i(\sigma)$. We implement this via a feasibility mask $m_{k,i}(\sigma_{<i}) \in \{0, -\infty\}$ added to the predicted scores:

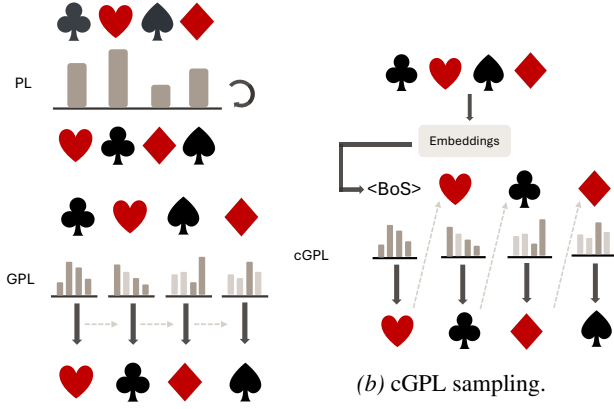
$$m_{k,i}(\sigma_{<i}) = \begin{cases} 0, & k \in \mathcal{R}_i(\sigma), \\ -\infty, & k \notin \mathcal{R}_i(\sigma), \end{cases} \quad (12)$$

$$\tilde{s}_{k,i} := s_{k,i}(X, \sigma_{<i}, t) + m_{k,i}(\sigma_{<i}), \quad (13)$$

so that $\text{softmax}(\tilde{s}_{:,i})$ renormalizes over $\mathcal{R}_i(\sigma)$ and assigns zero probability to infeasible items.

Training objective. We train our model by maximizing the standard variational lower bound:

$$\begin{aligned} & \mathbb{E}_{p_{\text{data}}(X_0, X)} [\log p_\theta(X_0 | X)] \\ & \geq \mathbb{E}_{p_{\text{data}}(X_0, X) q(X_{1:T} | X_0, X)} \left[\log p(X_T | X) \right. \\ & \quad \left. + \sum_{t=1}^T \log \frac{p_\theta(X_{t-1} | X_t, t)}{q(X_t | X_{t-1})} \right]. \end{aligned}$$



(a) PL and GPL sampling.

Figure 2. Sampling in PL/GPL/cGPL. (a) **PL and GPL.** In PL, each item is assigned a single scalar score; sampling a permutation amounts to repeatedly sampling from the same score vector without replacement, masking selected items and renormalizing at each step. In GPL, each item is assigned a length- N score vector, yielding position-specific logits: at step i we sample according to the i -th column of the score matrix (after masking previously selected items), proceeding sequentially from $i = 1$ to N . (b) **cGPL.** In cGPL, each item is assigned a position-dependent score vector *dynamically*. Sampling proceeds *autoregressively*: the score vector at later positions depends on the outcomes sampled at preceding positions. As in GPL, we apply masking and subsequent renormalization to obtain a valid probability distribution over the remaining items.

Our model predicts a denoised distribution $p_\theta(X_0 | X_t, t)$. To obtain a one-step reverse transition, we compose this predictor with a reconstruction kernel $\tilde{q}_t(X_{t-1} | X_t, X_0)$:

$$p_\theta(X_{t-1} | X_t, t) := \sum_{X_0} \tilde{q}_t(X_{t-1} | X_t, X_0) p_\theta(X_0 | X_t, t). \quad (14)$$

Note that $\tilde{q}_t$ in the discrete permutation space is generally intractable but as discussed in 3.2, we can augment with a tractable update step in the continuous soft-rank space.

With the feasibility mask in Eq 13, the negative log-likelihood reduces to a sum of cross-entropies on the masked scores:

$$\mathcal{L}(\sigma; X, t) = - \sum_{i=1}^N \log \frac{\exp(\tilde{s}_{\sigma(i), i})}{\sum_{k=1}^N \exp(\tilde{s}_{k, i})}. \quad (15)$$

Pointer parameterization. As an alternative to a fixed N -way linear head that directly outputs N logits (treating output coordinates as a fixed “vocabulary” indexed by absolute positions), we draw inspiration from Pointer Networks (Vinyals et al., 2017) and parameterize the cGPL scores with an item-aligned pointer head that explicitly *points* to the encoded input items.

Algorithm 2 Autoregressive Sampling from cGPL

Require: Input set $X = \{x_1, \dots, x_n\}$; diffusion step t ; neural decoder f_θ
Ensure: Permutation $\hat{\sigma} \in \mathcal{S}_n$

- 1: $\mathcal{R} \leftarrow \{1, 2, \dots, n\}$ (remaining items)
- 2: $\hat{\sigma} \leftarrow []$ (empty prefix)
- 3: **for** $i = 1, \dots, n$ **do**
- 4: Compute logits $s_{:,i} \leftarrow ([f_\theta(X, \hat{\sigma}_{<i}, t)]_{k,i})_{k=1}^n$ (mask)
- 5: $\tilde{s}_{k,i} \leftarrow s_{k,i}$ if $k \in \mathcal{R}$, else $-\infty$
- 6: $\pi_{:,i} \leftarrow \text{softmax}(\tilde{s}_{:,i})$
- 7: Sample $\hat{\sigma}(i) \sim \text{Categorical}(\pi_{:,i})$
- 8: $\mathcal{R} \leftarrow \mathcal{R} \setminus \{\hat{\sigma}(i)\}$
- 9: **end for**
- 10: **return** $\hat{\sigma}$

Concretely, at decoding step i we compute a compatibility score between the decoder state and each encoder item representation, and interpret the resulting length- N vector as a categorical distribution over the *input indices* (rather than over a fixed output dictionary); see Fig. 5 for an illustration.

This parameterization naturally supports variable-size output dictionaries and provides another way of producing valid permutations other than relying on versions of the Plackett-Luce distribution (Vinyals et al., 2017).

Formally, given encoder item representations and a prefix-dependent decoder state, we compute the pointer scores as a bi-affine transformation between them:

$$\begin{aligned} e_k &= \text{Enc}_\theta(X_t, t)_k \in \mathbb{R}^d, & k \in [N], \\ d_i &= \text{Dec}_\theta(\sigma_{<i}, X_t, t)_i \in \mathbb{R}^d, & i \in [N], \\ s_{k,i}(X_t, \sigma_{<i}, t) &= d_i^\top W e_k + u^\top d_i + v^\top e_k + b, \end{aligned} \quad (16)$$

where $W \in \mathbb{R}^{d \times d}$ and $u, v \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are learned parameters. The conditional distribution at step i is then obtained by applying the same remaining-set normalization as in Eq. (9).

Speeding up inference with KV caching. The cGPL/Pointer-cGPL decoders are autoregressive: at each diffusion timestep t , while the encoder-only GPL model needs only a single forward pass to compute the scores, our cGPL/Pointer-cGPL decoder must run N sequential forward passes, one per prefix position, to produce all stagewise scores. Without optimization, each of these passes redundantly recomputes attention keys and values for previously emitted prefix tokens, leading to a substantial wall-clock overhead that grows with N . To remove this redundancy, we adopt *key/value (KV) caching* (Pope et al., 2023), a well-established technique in natural language modeling with autoregressive transformers, which we can apply straightforwardly in our cGPL and Pointer-cGPL

decoders. We quantify the resulting inference speedup in Section 4. Note that this overhead is specific to inference: during training, teacher forcing makes the ground-truth prefix available at every position, so the N stagewise score vectors are computed in parallel within a single forward pass.

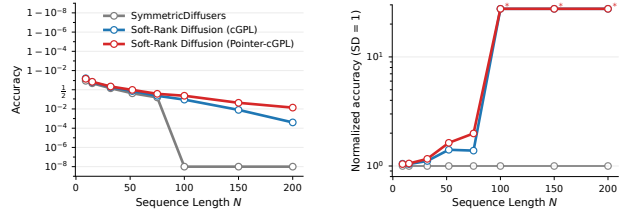
4. Experiments

4.1. 4-digit MNIST sorting

We evaluate Soft-Rank Diffusion on four-digit MNIST sorting, a standard benchmark for permutation modeling in differentiable sorting and permutation diffusion (Grover et al., 2019; Petersen et al., 2022; Zhang et al., 2025). Each instance is a list of N four-digit MNIST images (randomly ordered), and the task is to predict the permutation that sorts the list in ascending order by the underlying four-digit numbers they represent. We report *Kendall–Tau correlation* (a correlation coefficient between ranks) (Kendall, 1938), *accuracy* (fraction of exactly matched sorted lists), and *correctness* (fraction of items placed in their correct positions). We rerun the baseline methods DiffSort (Petersen et al., 2022), Error-free DiffSort (Kim et al., 2024) and SymmetricDiffusers (Zhang et al., 2025) using their official GitHub repositories and the recommended (or default) hyperparameters. For SymmetricDiffusers, we use GPL sampling and disable beam search so that the comparison focuses on cGPL versus GPL under the same sampling setting. See Appendix A.1 for details.

Table 1 summarizes results on the 4-digit MNIST sorting benchmark across sequence lengths $N \in \{9, 15, 32, 52, 75, 100, 150, 200\}$. Overall, Soft-Rank Diffusion consistently outperforms all baseline methods, with the performance gap widening as N increases. In particular, the Pointer variant further outperforms the plain cGPL variant, and is the only method that maintains non-trivial performance in the long-sequence regime ($N \geq 150$). While SymmetricDiffusers performs strongly on short sequences, its accuracy and correctness degrade rapidly as N grows, approaching chance-level behavior at $N > 100$. By contrast, Soft-Rank Diffusion degrades more gracefully, indicating improved scalability and robustness to longer permutations.

This scaling advantage is further illustrated in Figure 3, which examines exact-match accuracy as a function of the sequence length N . Figure 3a shows that both variants of Soft-Rank Diffusion are substantially less prone to catastrophic accuracy collapse as N increases, with the Pointer variant exhibiting consistently stronger robustness at larger sequence lengths. Figure 3b further quantifies this effect by reporting the relative exact-match accuracy of Soft-Rank Diffusion over SymmetricDiffusers, computed as the ratio relative to the baseline accuracy. The improvement margin



(a) Exact-match accuracy in logit scale over N .

(b) Accuracy ratio relative to SymmetricDiffusers (SD), shown on a log scale.

Figure 3. Exact-match accuracy and relative improvement versus SymmetricDiffusers as a function of sequence length (N). Panel 3a shows accuracy on a logit scale. Panel 3b reports the accuracy ratio of Soft-Rank Diffusion relative to SymmetricDiffusers on a log scale. Starred points mark sequence lengths where SymmetricDiffusers attains zero accuracy; the corresponding ratios are undefined and are clipped to the top of the plotted range for visualization.

grows rapidly with N , indicating that Soft-Rank Diffusion achieves increasingly favorable scaling behavior compared to SymmetricDiffusers.

4.2. Traveling Salesperson Problem (TSP)

We evaluate Soft-Rank Diffusion on the traveling salesperson problem (TSP), an NP-hard combinatorial optimization task. Given a set of 2D points $V = \{v_1, \dots, v_N\} \subset \mathbb{R}^2$, the goal is to predict a tour permutation $\sigma \in S_N$ that minimizes the total length $\sum_{i=1}^N \|v_{\sigma(i)} - v_{\sigma(i+1)}\|_2$, where $\sigma(N+1) := \sigma(1)$. We report the tour length and the optimality gap (see Appendix A.2 for details) on TSP-20 and TSP-50 ($N = 20$ and $N = 50$, respectively). We rerun SymmetricDiffusers using their respective official GitHub repositories and default hyper-parameters. For SymmetricDiffusers, we again use GPL sampling and disable beam search to ensure a fair comparison. See Appendix A.2

This task is particularly well-suited to highlight the advantage of our *dynamic* cGPL parameterization: unlike static scoring schemes, TSP decoding requires making prefix-dependent decisions over a shrinking set of remaining cities, where the desirability of each candidate depends strongly on the current partial tour. By producing context-conditioned, step-specific logits over the remaining items, cGPL better matches this dynamic decision structure and therefore exhibits larger performance gains as problem size increases.

Table 2 reports Euclidean TSP results on TSP-20 and TSP-50. Across both problem sizes, Soft-Rank Diffusion substantially improves over SymmetricDiffusers, reducing tour length and closing the optimality gap by more than two orders of magnitude. Moreover, the Pointer variant consistently achieves the best performance, yielding further gains over the fixed variant—especially on TSP-50—highlighting

Table 1. MNIST sorting benchmark. We compare baselines and our models across sequence lengths. Bold indicates the best value in each column. For a controlled comparison, SymmetricDiffusers and both Soft-Rank Diffusion variants use the same 7-layer Transformer backbone (ours: 3-layer encoder + 4-layer decoder with matched hidden width). This differs from Zhang et al. (2025), which used a 12-layer Transformer for $n = 200$ and a 7-layer Transformer for other lengths.

Method	Metric	Sequence Length N							
		9	15	32	52	75	100	150	200
DiffSort (Petersen et al., 2022)	Kendall-Tau $\uparrow$	0.8163	0.7386	0.4423	0.3037	0.2227	0.1617	0.0891	0.0636
	Accuracy $\uparrow$	0.5555	0.2350	0.0003	0.0000	0.0000	0.0000	0.0000	0.0000
	Correctness $\uparrow$	0.8643	0.7975	0.5335	0.3897	0.2987	0.2251	0.1307	0.0953
Error-free DiffSort (Kim et al., 2024)	Kendall-Tau $\uparrow$	0.9151	0.9160	0.8798	0.0042	0.0005	0.0029	0.0006	0.0002
	Accuracy $\uparrow$	0.7953	0.7492	0.5186	0.0000	0.0000	0.0000	0.0000	0.0000
	Correctness $\uparrow$	0.9365	0.9334	0.9019	0.0258	0.0132	0.0112	0.0072	0.0030
SymmetricDiffusers with GPL (Zhang et al., 2025)	Kendall-Tau $\uparrow$	0.9483	0.9320	0.8610	0.7890	0.7304	0.1114	0.0641	0.0022
	Accuracy $\uparrow$	0.8955	0.8260	0.5896	0.3009	0.1379	0.0000	0.0000	0.0000
	Correctness $\uparrow$	0.9605	0.9450	0.8846	0.8232	0.7733	0.1620	0.0975	0.0061
Soft-Rank Diffusion with cGPL Ours	Kendall-Tau $\uparrow$	0.9668	0.9384	0.8765	0.8092	0.6925	0.6279	0.4821	0.3982
	Accuracy $\uparrow$	0.9397	0.8601	0.6555	0.4239	0.1908	0.0863	0.0080	0.0004
	Correctness $\uparrow$	0.9742	0.9505	0.8967	0.8393	0.7347	0.6773	0.5468	0.4644
Soft-Rank Diffusion with Pointer-cGPL Ours	Kendall-Tau $\uparrow$	0.9645	0.9455	0.8944	0.8459	0.7727	0.7454	0.6198	0.6048
	Accuracy $\uparrow$	0.9329	0.8710	0.6854	0.4909	0.2742	0.1922	0.0421	0.0137
	Correctness $\uparrow$	0.9719	0.9559	0.9119	0.8724	0.8082	0.7849	0.6704	0.6607

Table 2. TSP performance on TSP-20 and TSP-50. We report tour length $L(\text{Tour})$ and the optimality gap (both lower is better).

Method	TSP-20		TSP-50	
	$L(\text{Tour}) \downarrow$	Gap $\downarrow$	$L(\text{Tour}) \downarrow$	Gap $\downarrow$
SymmetricDiffusers with GPL (Zhang et al., 2025)	5.4342	0.4169	12.8525	1.2618
Soft-Rank Diffusion with cGPL Ours	3.8732	0.0079	5.7336	0.0078
Soft-Rank Diffusion with Pointer-cGPL Ours	3.8557	0.0034	5.7157	0.0047

the benefit of our dynamic cGPL parameterization for prefix-dependent decoding over a shrinking candidate set.

4.3. CIFAR-10 Jigsaw

We evaluate Soft-Rank Diffusion on the CIFAR-10 (Krizhevsky, 2009) jigsaw puzzle task as a third benchmark complementing MNIST sorting and TSP. A CIFAR-10 image is divided into an $n \times n$ grid of patches ($N = n^2$) presented in random order, and the model must predict the permutation that reconstructs the original raster ordering. Unlike sorting (element-wise) or TSP (sequential), jigsaw reconstruction requires global spatial reasoning over all patches: placing a patch depends on the full image context. We compare against SymmetricDiffusers (Zhang et al., 2025) at three scales, 3×3 , 4×4 , and 8×8 , using the same metrics as in the MNIST sorting experiments.

Table 3 reports the results. The qualitative pattern matches sorting and TSP: as N grows, the gap between Soft-Rank Diffusion and SymmetricDiffusers widens, and at 8×8 the discrete baseline collapses to chance-level behavior while Soft-Rank Diffusion continues to learn meaningful structure. The decoder comparison at 8×8 further shows the Pointer variant substantially outperforming vanilla cGPL, consistent with the intuition that cross-attention over all positions becomes critical when the permutation space is large.

Failure mode analysis. Figure 4 examines how each method fails at 8×8 . The discrete baseline makes predictions that are uniformly near chance across all positions, indicating that no meaningful positional structure has been learned. The cGPL variant exhibits a top-left to bottom-right accuracy gradient consistent with autoregressive error accumulation, while the Pointer variant maintains high accuracy across all positions with a much flatter gradient, owing to its biaffine cross-attention head attending to all positions simultaneously. Patch-wise displacement distributions (Fig. 7) reinforce this picture: Soft-Rank Diffusion concentrates errors near the correct position—even when wrong, patches typically land within a small displacement—whereas SymmetricDiffusers produces an approximately random distribution. We attribute the contrast to the locality induced by the continuous forward process: small noise in $[0, 1]^N$ swaps nearby soft ranks, so the reverse process naturally resolves local ambiguities before global ones, while discrete riffle-shuffle dynamics perform global mixing at each step and leave no local structure for the reverse process to exploit.

Table 3. **CIFAR-10 jigsaw benchmark.** Bold marks the best value per (metric, n). All methods use a 7-layer Transformer backbone with the same hidden width.

Method	Metric	Grid Size $n \times n$		
		3×3 ($N=9$)	4×4 ($N=16$)	8×8 ($N=64$)
SymmetricDiffusers with GPL (Zhang et al., 2025)	Kendall-Tau $\uparrow$	0.8312	0.6101	0.0250
	Accuracy $\uparrow$	0.7002	0.2172	0.0000
	Correctness $\uparrow$	0.8662	0.6753	0.0501
Soft-Rank Diffusion with cGPL Ours	Kendall-Tau $\uparrow$	0.9494	0.9320	0.2521
	Accuracy $\uparrow$	0.9221	0.8752	0.0924
	Correctness $\uparrow$	0.9564	0.9413	0.2854
Soft-Rank Diffusion with Pointer-cGPL Ours	Kendall-Tau $\uparrow$	0.9542	0.9345	0.4962
	Accuracy $\uparrow$	0.9258	0.8681	0.2700
	Correctness $\uparrow$	0.9601	0.9448	0.5256

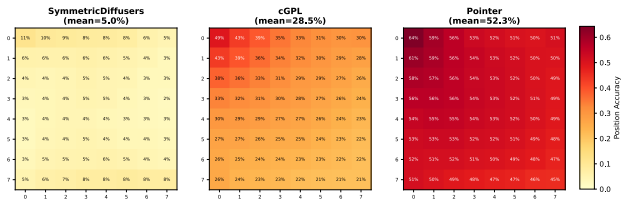


Figure 4. **Per-position accuracy heatmaps on 8×8 CIFAR-10 jigsaw.** Each cell is the fraction of test samples where that grid position is correctly assigned. Left: SymmetricDiffusers (mean 5.0%). Middle: cGPL (mean 28.5%). Right: Pointer-cGPL (mean 52.3%). Random baseline $1/64 \approx 1.6\%$.

Runtime and KV cache. Table 9 reports wall-clock inference time. With KV cache enabled, Soft-Rank Diffusion is comparable to SymmetricDiffusers at small N but slower at large N due to the autoregressive cGPL decode; the additional runtime at 8×8 nevertheless buys meaningful accuracy in a regime where SymmetricDiffusers achieves none, as demonstrated in Figure 6. The KV-cache speedup grows with N , reflecting the per-prefix-position savings discussed in Section 3.3.

5. Ablation Studies

We study the impact of several key design choices of Soft-Rank Diffusion in Table 8. While our default configuration adopts an σ_0 parametrization for the cGPL reverse model, we observe that the model achieves consistently strong performance when using an σ_{t-1} parametrization, as also employed in Zhang et al. (2025). In contrast, when paired with the Riffle Shuffle forward process, the σ_0 parametrization consistently fails, leading to a severe degradation in performance across all evaluation metrics. This may be due to the intractability of the reverse transition when the forward process is defined via a riffle shuffle. Moreover, even under the shared σ_{t-1} parametrization, models trained with the Soft-Rank Diffusion forward process consistently outperform those using the Riffle Shuffle forward process.

Finally, incorporating a biaffine pointer mechanism in the cGPL reverse model further improves performance over the vanilla cGPL variant, highlighting the benefit of explicitly modeling autoregressive selection in the reverse dynamics.

6. Conclusions

We presented Soft-Rank Diffusion, a diffusion framework for permutation-valued data that bridges diffusion modeling with principled ranking distributions. By lifting permutations into a continuous latent space through soft rank representations, our approach enables a smoother and more tractable forward process than existing abrupt schemes such as riffle-shuffles, and a theoretically sound reverse solver grounded in Plackett-Luce-style likelihoods. Central to our method is the proposed contextualized Generalized Plackett-Luce (cGPL) parameterization, which conditions each selection step on the evolving prefix and generalizes prior prefix-agnostic models.

Across sorting and combinatorial optimization benchmarks, Soft-Rank Diffusion consistently outperforms existing permutation diffusion and differentiable sorting approaches, with particularly strong gains on long and intrinsically sequential permutations. These results highlight the importance of context-aware reverse dynamics for scalable permutation modeling. We believe this framework opens the door to further unifying diffusion-based generative modeling with structured, likelihood-based formulations over discrete combinatorial objects.

Impact Statement

This paper advances methods in machine learning. We do not anticipate societal impacts that require specific discussion beyond standard considerations.

References

- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces, 2023. URL <https://arxiv.org/abs/2107.03006>.
- Mathieu Blondel, Olivier Teboul, Quentin Berthet, and Josip Djolonga. Fast differentiable sorting and ranking, 2020. URL <https://arxiv.org/abs/2002.08871>.
- Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models, 2022. URL <https://arxiv.org/abs/2205.14987>.
- Marco Cuturi, Olivier Teboul, and Jean-Philippe Vert. Differentiable ranks and sorting using optimal transport, 2019. URL <https://arxiv.org/abs/1905.11885>.
- Sander Dieleman, Laurent Sartran, Arman Roshannai, Nikolay Savinov, Yaroslav Ganin, Pierre H. Richemond, Arnaud Doucet, Robin Strudel, Chris Dyer, Conor Durkan, Curtis Hawthorne, Rémi Leblond, Will Grathwohl, and Jonas Adler. Continuous diffusion for categorical data, 2022. URL <https://arxiv.org/abs/2211.15089>.
- Yufei Feng, Yu Gong, Fei Sun, Junfeng Ge, and Wenwu Ou. Revisit recommender system in the permutation prospective, 2021. URL <https://arxiv.org/abs/2102.12057>.
- Itai Gat, Tal Remez, Neta Shaul, Felix Kreuk, Ricky T. Q. Chen, Gabriel Synnaeve, Yossi Adi, and Yaron Lipman. Discrete flow matching, 2024. URL <https://arxiv.org/abs/2407.15595>.
- Edgar Gilbert. Theory of shuffling. Technical report, Technical memorandum, Bell Laboratories, 1955.
- Aditya Grover, Eric Wang, Aaron Zweig, and Stefano Ermon. Stochastic optimization of sorting networks via continuous relaxations, 2019. URL <https://arxiv.org/abs/1903.08850>.
- Ishaan Gulrajani and Tatsunori B. Hashimoto. Likelihood-based diffusion language models, 2023. URL <https://arxiv.org/abs/2305.18619>.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.
- Chaitanya K. Joshi, Quentin Cappart, Louis-Martin Rousseau, and Thomas Laurent. Learning TSP requires rethinking generalization. In *Principles and Practice of Constraint Programming (CP 2021)*, volume 210, pages 33:1–33:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021. doi: 10.4230/LIPICS.CP.2021.33. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPICS.CP.2021.33>.
- M. G. Kendall. A new measure of rank correlation. *Biometrika*, 30(1-2):81–93, 06 1938. ISSN 0006-3444. doi: 10.1093/biomet/30.1-2.81. URL <https://doi.org/10.1093/biomet/30.1-2.81>.
- Jungtaek Kim, Jeongbeen Yoon, and Minsu Cho. Generalized neural sorting networks with error-free differentiable swap functions, 2024. URL <https://arxiv.org/abs/2310.07174>.
- Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. doi: 10.1109/MC.2009.263.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- H. W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1-2):83–97, 1955. doi: <https://doi.org/10.1002/nav.3800020109>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1002/nav.3800020109>.
- Tie-Yan Liu. Learning to rank for information retrieval. *Found. Trends Inf. Retr.*, 3(3):225–331, March 2009. ISSN 1554-0669. doi: 10.1561/15000000016. URL <https://doi.org/10.1561/15000000016>.
- Aaron Lou and Stefano Ermon. Reflected diffusion models, 2023. URL <https://arxiv.org/abs/2304.04740>.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution, 2024. URL <https://arxiv.org/abs/2310.16834>.
- R. D. Luce. *Individual Choice Behavior*. John Wiley, 1959.
- TorchVision maintainers and contributors. TorchVision: PyTorch’s computer vision library. <https://github.com/pytorch/vision>, 2016.
- Gonzalo Mena, David Belanger, Scott Linderman, and Jasper Snoek. Learning latent permutations with Gumbel-Sinkhorn networks, 2018. URL <https://arxiv.org/abs/1802.08665>.

- Felix Petersen, Christian Borgelt, Hilde Kuehne, and Oliver Deussen. Monotonic differentiable sorting networks, 2022. URL <https://arxiv.org/abs/2203.09630>.
- R. L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 24(2):193–202, 1975. doi: <https://doi.org/10.2307/2346567>. URL <https://rss.onlinelibrary.wiley.com/doi/abs/10.2307/2346567>.
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling Transformer inference. In Dawn Song, Michael Carbin, and Tianqi Chen, editors, *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*. mlsys.org, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/hash/c4be71ab8d24cdfb45e3d06dbfca2780-Abstract-mlsys2023.html.
- Sebastian Prillo and Julian Martin Eisenschlos. SoftSort: A continuous relaxation for the argsort operator, 2020. URL <https://arxiv.org/abs/2006.16038>.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data, 2025. URL <https://arxiv.org/abs/2406.04329>.
- Richard Sinkhorn. A relationship between arbitrary positive matrices and doubly stochastic matrices. *The Annals of Mathematical Statistics*, 35(2):876–879, 1964. ISSN 00034851. URL <http://www.jstor.org/stable/2238545>.
- Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics, 2015. URL <https://arxiv.org/abs/1503.03585>.
- Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models, 2022. URL <https://arxiv.org/abs/2010.02502>.
- Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations, 2021. URL <https://arxiv.org/abs/2011.13456>.
- Haoran Sun, Lijun Yu, Bo Dai, Dale Schuurmans, and Hanjun Dai. Score-based continuous-time discrete diffusion models, 2023. URL <https://arxiv.org/abs/2211.16750>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- Clement Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. DiGress: Discrete denoising diffusion for graph generation, 2023. URL <https://arxiv.org/abs/2209.14734>.
- Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks, 2017. URL <https://arxiv.org/abs/1506.03134>.
- Tianyu Xie, Yu Zhu, Longlin Yu, Tong Yang, Ziheng Cheng, Shiyue Zhang, Xiangyu Zhang, and Cheng Zhang. Reflected flow matching, 2024. URL <https://arxiv.org/abs/2405.16577>.
- Yongxing Zhang, Donglin Yang, and Renjie Liao. SymmetricDiffusers: Learning discrete diffusion on finite symmetric groups, 2025. URL <https://arxiv.org/abs/2410.02942>.
- Linqi Zhou, Aaron Lou, Samar Khanna, and Stefano Ermon. Denoising diffusion bridge models, 2023. URL <https://arxiv.org/abs/2309.16948>.

A. Details on experimental setup

A.1. 4-digit MNIST sorting

Dataset Following Zhang et al. (2025), we generate the 4-digit MNIST dataset using the code from their official GitHub repository. For each experiment, we construct training sequences by sampling 60,000 length- N sequences of 4-digit MNIST images from `torchvision.datasets.MNIST` (maintainers and contributors, 2016), where N is an experiment-specific parameter. We generate an additional 10,000 sequences for testing. Each 4-digit image is created on the fly by first sampling four digits uniformly at random, then drawing the corresponding digit images from MNIST, and finally concatenating them into a single 4-digit image.

Model architecture For SymmetricDiffusers, we use a 7-layer encoder-only Transformer with hidden dimension 512, feedforward dimension 128, with 8 attention heads. For Soft-Rank Diffusion, we match the overall model capacity by using a 7-layer encoder-decoder Transformer, with 3 layers in the encoder and 4 layers in the decoder. We keep the hidden and feedforward dimensions the same as SymmetricDiffusers. **Note:** While Zhang et al. (2025) use a 12-layer Transformer for $N=200$, we use a 7-layer Transformer for all N to keep model capacity consistent across sequence lengths and ensure performance is comparable across settings.

Training All models are trained on a single NVIDIA H100 GPU with batch size 64 for 120 epochs. Training data are generated on the fly at each epoch. For SymmetricDiffusers, we use the default hyperparameters from the authors’ official GitHub repository. For $N \in 75, 150$, where no default hyperparameters are provided, we reuse the settings for $N = 52$ and $N = 100$, respectively.

Inference and metrics At inference time, we randomly generate 10,000 sequences of N 4-digit MNIST images. Since each image is labeled by its underlying 4-digit number, we can obtain the ground-truth ascending order directly from the labels. We report the same three metrics as Zhang et al. (2025): Kendall-Tau coefficient, Accuracy (exact match), and Correctness (element-wise match). Kendall-Tau measures rank correlation between the predicted and ground-truth permutations; Accuracy indicates whether the two permutations are identical; and Correctness is the fraction of elements placed in the correct position.

Non-diffusion model baselines We retrain and evaluate DiffSort (Petersen et al., 2022) and Error-free DiffSort (Kim et al., 2024) under the same dataset setup. For DiffSort, we use the default hyperparameters from the authors’ official GitHub repository, with the odd-even sorting network, steepness 10, and learning rate $10^{-3.5}$. For Error-free DiffSort, we use TransformerL as the backbone and the hyperparameters listed in Table 4.

Table 4. Hyperparameters for Error-Free DiffSort on 4-digit MNIST sorting.

Sequence Length	Steepness	Sorting Network	Loss Weight	Learning Rate
9	34	odd even	1.00	10^{-4}
15	25	odd even	0.10	10^{-4}
32	124	odd even	0.10	10^{-4}
52	130	bitonic	0.10	$10^{-3.5}$
75	135	bitonic	0.10	$10^{-3.5}$
100	140	bitonic	0.10	$10^{-3.5}$
150	170	bitonic	0.10	$10^{-3.5}$
200	200	bitonic	0.10	10^{-4}

A.2. TSP

Dataset For TSP-20 and TSP-50, we use the dataset from Zhang et al. (2025), which is adapted from Joshi et al. (2021). The training set contains 1,512,000 graphs, and the test set contains 1,280 graphs.

Model architecture For SymmetricDiffusers, we use a 16-layer encoder-only Transformer with hidden dimension 1024, feedforward dimension 256, with 8 attention heads. For Soft-Rank Diffusion, we match the overall model capacity by using

a 16-layer encoder–decoder Transformer, with 8 layers in the encoder and 8 layers in the decoder. We keep the hidden and feedforward dimensions the same as SymmetricDiffusers.

Training All models are trained on a single NVIDIA H100 GPU for 50 epochs with batch size 64. We use a peak learning rate of 2×10^{-4} with a cosine decay schedule and 51,600 warm-up steps.

Inference and metrics At inference time, we evaluate on the 1,280 held-out test graphs and report two metrics: the tour length of the predicted solution, L_{pred} , and the optimality gap relative to an OR solver baseline, L_{OR} . The optimality gap is defined as

$$\text{Gap} = \frac{L_{\text{pred}} - L_{\text{OR}}}{L_{\text{OR}}}$$

B. Derivations

Here we derive the reverse conditional transition kernel $q(z_s | z_t, z_0, z_1)$ for $0 < s < t < 1$ in Eq. 6. We present the derivation for a single scalar soft-rank coordinate; the N -dimensional extension follows by independence across coordinates.

For the (unreflected) Brownian bridge with noise scale η , the marginal at time $u \in (0, 1)$ is Gaussian:

$$q(z_u | z_0, z_1) = \mathcal{N}(z_u; \mu(u), \eta^2 u(1-u)), \quad \mu(u) := (1-u)z_0 + uz_1. \quad (17)$$

A convenient representation of a Brownian bridge is

$$z_u = \mu(u) + \eta b_u, \quad u \in [0, 1], \quad (18)$$

where b_u is a standard Brownian bridge with $b_0 = b_1 = 0$ and covariance $\text{Cov}(b_s, b_t) = \min(s, t) - st$. Therefore, conditional on (z_0, z_1) , the pair (z_s, z_t) is jointly Gaussian with means

$$\mathbb{E}[z_s | z_0, z_1] = \mu(s), \quad \mathbb{E}[z_t | z_0, z_1] = \mu(t), \quad (19)$$

variances

$$v_s := \text{Var}(z_s | z_0, z_1) = \eta^2 s(1-s), \quad v_t := \text{Var}(z_t | z_0, z_1) = \eta^2 t(1-t), \quad (20)$$

and covariance (for $s < t$)

$$c_{st} := \text{Cov}(z_s, z_t | z_0, z_1) = \eta^2 (\min(s, t) - st) = \eta^2 s(1-t). \quad (21)$$

Hence, writing $\mu_s := \mu(s)$, $\mu_t := \mu(t)$ and $c := c_{st}$,

$$q(z_s, z_t | z_0, z_1) = \mathcal{N}\left(\begin{bmatrix} z_s \\ z_t \end{bmatrix}; \begin{bmatrix} \mu_s \\ \mu_t \end{bmatrix}, \begin{bmatrix} v_s & c \\ c & v_t \end{bmatrix}\right), \quad q(z_t | z_0, z_1) = \mathcal{N}(z_t; \mu_t, v_t). \quad (22)$$

By Bayes' rule,

$$q(z_s | z_t, z_0, z_1) = \frac{q(z_s, z_t | z_0, z_1)}{q(z_t | z_0, z_1)}. \quad (23)$$

Since the denominator does not depend on z_s (for fixed z_t), it suffices to view the joint density as a function of z_s and complete the square. Let $x := z_s - \mu_s$ and $y := z_t - \mu_t$. Using the closed-form inverse for a symmetric 2×2 matrix, the inverse of the covariance matrix is then

$$\begin{bmatrix} v_s & c \\ c & v_t \end{bmatrix}^{-1} = \frac{1}{D} \begin{bmatrix} v_t & -c \\ -c & v_s \end{bmatrix}, \quad D := v_s v_t - c^2. \quad (24)$$

Therefore,

$$\begin{aligned} \begin{bmatrix} x \\ y \end{bmatrix}^\top \begin{bmatrix} v_s & c \\ c & v_t \end{bmatrix}^{-1} \begin{bmatrix} x \\ y \end{bmatrix} &= \frac{1}{D} (v_t x^2 - 2cxy + v_s y^2) \\ &= \frac{v_t}{D} \left(x - \frac{c}{v_t} y\right)^2 + \text{const}(y), \end{aligned} \quad (25)$$

where $\text{const}(y)$ does not depend on x (hence not on z_s). This yields the Gaussian conditional

$$z_s \mid z_t, z_0, z_1 \sim \mathcal{N}\left(\mu_s + \frac{c}{v_t}(z_t - \mu_t), v_s - \frac{c^2}{v_t}\right). \quad (26)$$

Substituting $v_s = \eta^2 s(1-s)$, $v_t = \eta^2 t(1-t)$, and $c = \eta^2 s(1-t)$ (for $s < t$), we obtain

$$\frac{c}{v_t} = \frac{s}{t}, \quad v_s - \frac{c^2}{v_t} = \eta^2 \frac{s(t-s)}{t}. \quad (27)$$

Hence,

$$z_s \mid z_t, z_0, z_1 \sim \mathcal{N}\left(\mu(s) + \frac{s}{t}(z_t - \mu(t)), \eta^2 \frac{s(t-s)}{t}\right), \quad 0 < s < t < 1. \quad (28)$$

Extension to N dimensions. Under coordinate-wise independent bridges sharing the same noise scale η , the same conditional holds with $\epsilon \sim \mathcal{N}(0, I_N)$, i.e., the conditional covariance becomes $\eta^2 \frac{s(t-s)}{t} I_N$.

C. Additional Analysis

C.1. Sensitivity to the initial reference distribution

A natural design question for Soft-Rank Diffusion is the choice of the reference distribution p_{ref} used as the endpoint of the forward Brownian bridge (Eq. 4). Two simple choices are the unit-cube uniform $\text{Unif}([0, 1]^N)$ (matching the natural domain of soft ranks) and the standard Gaussian $\mathcal{N}(0, I_N)$ (matching the convention from continuous-space diffusion models in $\mathbb{R}^N$). We ablate this choice on CIFAR-10 jigsaw with 4×4 ($N = 16$) for both the cGPL and Pointer-cGPL decoders.

Matched train and evaluation reference. We first compare the two reference choices in the *matched* setting, where training and evaluation use the same p_{ref} (Table 5). Performance is essentially identical across uniform and Gaussian choices for both decoders (within 0.5 percentage points of exact-match accuracy and 0.002 in Kendall τ), indicating that under matched train/evaluation, the choice of p_{ref} is not a critical hyperparameter.

Table 5. Matched train/eval p_{ref} on CIFAR10 jigsaw ($n = 4$, $\sigma = 0.5$, $T = 5$). Both Uniform and Gaussian reference distributions yield essentially identical performance, indicating the choice of p_{ref} has negligible impact on training.

Decoder	Init Distribution	Perm Acc (%)	Kendall τ	Prop Correct (%)
cGPL	Uniform	86.67	0.9303	93.93
	Gaussian	86.96	0.9317	94.08
Pointer-cGPL	Uniform	87.35	0.9365	94.43
	Gaussian	86.94	0.9347	94.38

Mismatched train and evaluation reference. A stronger test of robustness is to deliberately *mismatch* the training and evaluation reference distributions—training under one distribution and evaluating under the other (Table 6). All four train/eval combinations yield essentially the same accuracy (within noise).

Table 6. Mismatched train/eval p_{ref} on CIFAR10 jigsaw ($n = 4$, $\sigma = 0.5$, $T = 5$). Each cell shows Perm Acc / Prop Correct / Kendall τ . Performance is preserved even when training and evaluation use different reference distributions, confirming the Brownian bridge formulation drives z toward z_0 regardless of starting distribution.

Decoder	Train p_{ref}	Eval: Uniform	Eval: Gaussian
cGPL	Uniform	86.67 / 93.93 / $\tau = 0.930$	87.09 / 94.08 / $\tau = 0.931$
	Gaussian	86.71 / 94.01 / $\tau = 0.932$	86.96 / 94.08 / $\tau = 0.932$
Pointer-cGPL	Uniform	87.35 / 94.43 / $\tau = 0.937$	87.33 / 94.51 / $\tau = 0.937$
	Gaussian	87.17 / 94.31 / $\tau = 0.935$	86.94 / 94.38 / $\tau = 0.935$

We adopt $\text{Unif}([0, 1]^N)$ in the main body of the paper for simplicity (matching the natural soft-rank domain), but the results above show that this choice is not critical to the performance of Soft-Rank Diffusion, and practitioners can swap in $\mathcal{N}(0, I_N)$ or other reasonable references without affecting performance.

C.2. Discussion on alternative choices of relaxation space

A natural perspective on permutation generation is to relax *permutation matrices* $P_\sigma \in \{0, 1\}^{N \times N}$ rather than ranks. The two most common matrix-based relaxations are the **Birkhoff polytope** B_N (the convex hull of permutation matrices, i.e., doubly stochastic matrices) (Mena et al., 2018; Cuturi et al., 2019) and the **orthogonal group** $O(N)$. We chose instead to relax the rank vector to $[0, 1]^N$ for both representational and computational reasons. Table 7 contrasts the three spaces.

Table 7. Comparison of three continuous relaxations of permutations. The soft-rank relaxation $[0, 1]^N$ used by Soft-Rank Diffusion has linear dimension and admits independent closed-form Brownian-bridge dynamics, while Birkhoff- and $O(N)$ -based relaxations require coupled, more expensive constructions.

	$[0, 1]^N$ (ours)	Birkhoff polytope B_N	Orthogonal group $O(N)$
Dimension	N	$(N - 1)^2$	$N(N - 1)/2$
Dynamics	Independent reflected Brownian bridges with closed-form transition kernels	Coupled entries; requires per-step Sinkhorn projection	Riemannian geodesics
Decoding	argsort (Eq. 3)	Hungarian or Sinkhorn rounding	Nearest permutation matrix

One key practical advantage is the dimensionality scaling. Our representation requires only N scalars, whereas B_N requires $(N - 1)^2$ free parameters; at $N = 200$ this is a factor of $\sim 200\times$ (200 vs. 39,601 dimensions). This quadratic scaling makes matrix-based relaxations increasingly impractical for long sequences, both for the computational cost of the diffusion dynamics and for the modeling burden of the score / denoising network. Furthermore, the entries of a doubly stochastic matrix are constrained by row- and column-sum identities, so a diffusion process in B_N must either project back onto B_N at each step (e.g., via Sinkhorn iteration (Sinkhorn, 1964; Mena et al., 2018)) or operate in a tangent space; both incur per-step cost that is absent from our coordinate-wise reflected bridge.

We emphasize that this is a difference of *representation*, not a competing relaxation: rather than softening permutation matrices, we soften the rank vector itself, an equivalent encoding of the permutation that scales linearly in N and admits independent dynamics.

Extension to constrained permutations. A further benefit of working in $[0, 1]^N$ is that structural constraints on permutations translate into simple *geometric* constraints on the latent space: common combinatorial constraints (partial orders, forbidden assignments) reduce to half-space inequalities or coordinate masks. For example, the partial-order constraint “ a appears before b ” becomes the half-space $\{z \in [0, 1]^N : z_a < z_b\}$, which our reflected bridge preserves by adding a single reflection plane, whereas the same constraint couples all N^2 entries in the Birkhoff polytope. We do not pursue constrained permutation generation experimentally and leave this direction to future work.

C.3. Chamber geometry of the soft-rank latent space

The decoding map $\sigma = \text{argsort}(\text{argsort}(Z))$ used by Soft-Rank Diffusion is *many-to-one*: many distinct latent vectors $Z \in [0, 1]^N$ induce the same permutation σ . This subsection elaborates on the geometry of this map and argues that the many-to-one structure is a useful feature of our parameterization rather than a limitation.

Chambers. The latent cube $[0, 1]^N$ is partitioned (up to a measure-zero boundary where coordinates tie) into $N!$ open convex *chambers*, one per permutation:

$$C_\sigma := \{z \in [0, 1]^N : z_{\sigma^{-1}(1)} < z_{\sigma^{-1}(2)} < \dots < z_{\sigma^{-1}(N)}\}. \quad (29)$$

The chambers $\{C_\sigma\}_{\sigma \in \mathcal{S}_N}$ are congruent (related by coordinate permutations) and partition the open cube; each has volume $1/N!$, so the uniform reference distribution $\text{Unif}([0, 1]^N)$ assigns equal mass to every permutation—a natural uninformative prior on $\mathcal{S}_N$. The LiftToGrid operator (Eq. 3) places the data latent Z_0 at the centroid of C_{σ_0} , the point of maximum distance $(1/(2N))$ to every chamber boundary, providing the most boundary-robust starting point for forward noising.

Many-to-one decoding. Three consequences of the chamber structure make $[0, 1]^N$ a more forgiving latent space than zero-dimensional vertex-based representations such as $\{P_\sigma\} \subset B_N$:

1. **Tolerance margin.** The reverse-time sampler only needs to place its prediction $\hat{z}_0$ *anywhere inside* the correct chamber C_{σ_0} —a positive-volume region. Contrast this with Birkhoff-based methods, where the targets P_σ are zero-dimensional vertices of B_N and correct recovery relies on a separate rounding procedure (e.g., the Hungarian algorithm (Kuhn, 1955)) to map nearby doubly stochastic matrices to the correct vertex. The chamber margin grants Soft-Rank Diffusion robustness to small errors in the denoising prediction.
2. **Confidence refinement.** Within a chamber, the reverse trajectory may move *toward the interior* (away from boundaries) without changing the decoded permutation. This provides a natural form of confidence calibration: after the chamber is selected, further reverse steps reduce boundary distance, increasing margin without altering the answer.
3. **Tie-breaking is measure-zero.** The score network outputs continuous values, so $\Pr(z_i = z_j) = 0$ almost surely under any continuous noise model. Decoding ambiguities at chamber boundaries therefore occur with probability zero, and the empirical predictions of our trained networks consistently maintain clear separation from boundaries.

Together, these properties suggest that the apparent “redundancy” of having $N!$ chambers map to the same set of $N!$ permutations is not a wasteful overparameterization, but a deliberate trade of zero-dimensional precision for positive-volume tolerance that the diffusion sampler can exploit.

D. Additional Figures and Tables

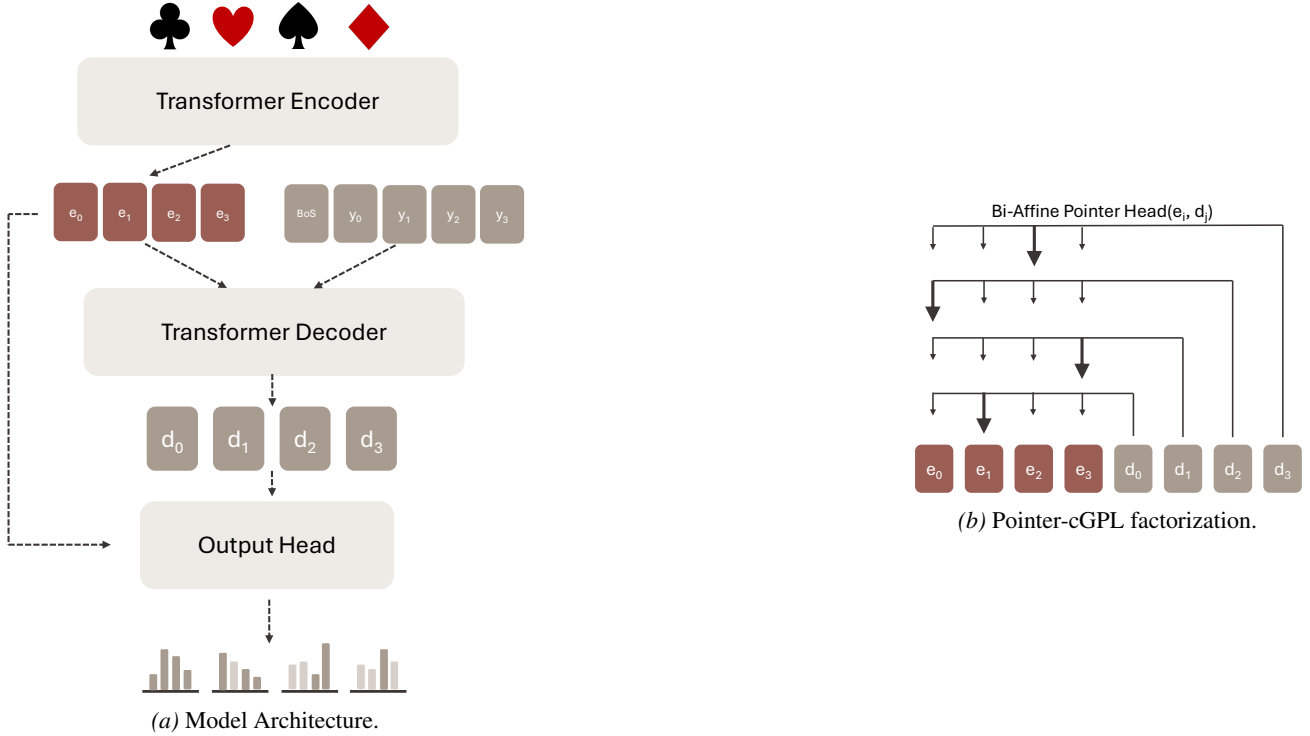


Figure 5. Model architecture and Pointer-cGPL parameterization for permutation generation. We adopt a standard encoder–decoder Transformer backbone (Vaswani et al., 2023). In the vanilla cGPL parameterization, the decoder states are mapped to logits via a linear output head (yielding a distribution over candidates at each step). In contrast, Panel 5b illustrates *Pointer-cGPL*, where a bi-affine compatibility module scores each encoded item representation against the current decoder state, producing step-wise logits over the input items that can be interpreted as a pointer distribution.

Table 8. Ablation study over forward diffusion processes, reverse models, and parametrizations on sorting 32 4-digit MNIST images.

Forward Process	Reverse Model	Parametrization	Kendall-Tau $\uparrow$	Accuracy $\uparrow$	Correctness $\uparrow$
Riffle Shuffle	GPL	x_{t-1}	0.8610	0.5896	0.8846
		x_0	-0.0013	0.0000	0.0312
Soft-Rank Diffusion	cGPL	x_{t-1}	0.8764	0.6425	0.8958
	cGPL w/ Biaffine Pointer	x_{t-1}	0.9074	0.7152	0.9232
Riffle Shuffle	cGPL	x_{t-1}	0.8527	0.5833	0.8758
	cGPL w/ Biaffine Pointer	x_{t-1}	0.8742	0.6404	0.8950
	cGPL	x_0	-0.0011	0.0000	0.0313
	cGPL w/ Biaffine Pointer	x_0	0.0001	0.0000	0.0314

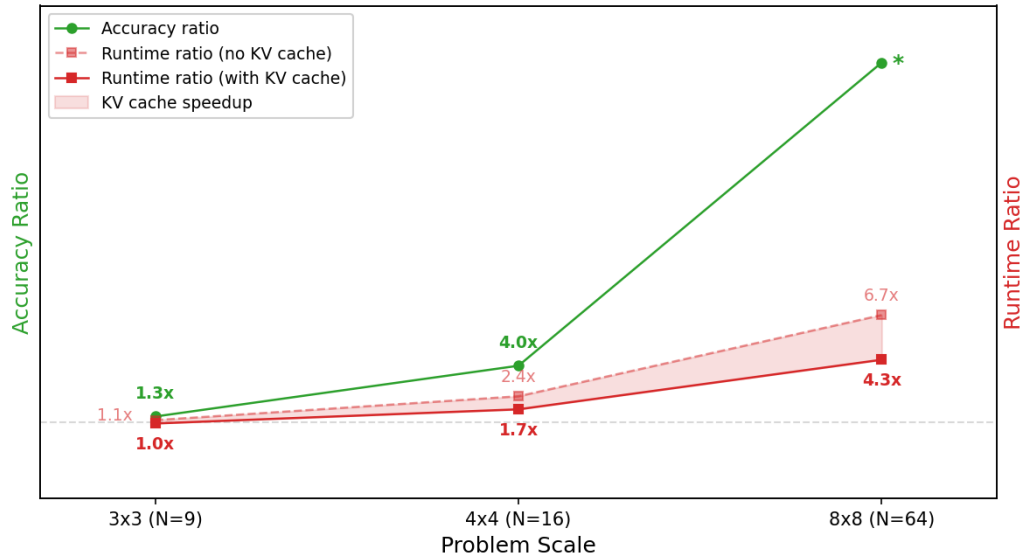


Figure 6. **Accuracy ratio** (green; Soft-Rank Pointer accuracy / SymmetricDiffusers accuracy) and **runtime ratio** (red; Soft-Rank Pointer wall-clock time / SymmetricDiffusers wall-clock time) across problem scales. The accuracy advantage grows superlinearly with N , reaching $4.0\times$ at $N=16$ and becoming unbounded at $N=64$ (*baseline accuracy is 0%). Runtime overhead remains moderate: with KV cache optimization (solid red), the slowdown is $1.0\times$ – $4.3\times$, while the shaded region shows the speedup across diffusion steps (up to $1.55\times$ at $N=64$). **The accuracy gains far outpace the runtime cost at every scale.**

 Table 9. **Wall-clock inference time.** Single H100 GPU, 10,000 test samples.

Method	3×3	4×4	8×8
SymmetricDiffusers	28.0s	21.6s	36.3s
cGPL, w/o cache	31.4s	41.5s	216.6s
cGPL, w/ cache	26.3s	31.4s	131.7s
<i>speedup</i>	<i>1.19×</i>	<i>1.32×</i>	<i>1.64×</i>
Pointer-cGPL, w/o cache	31.6s	51.6s	242.3s
Pointer-cGPL, w/ cache	26.7s	36.7s	156.7s
<i>speedup</i>	<i>1.18×</i>	<i>1.41×</i>	<i>1.55×</i>

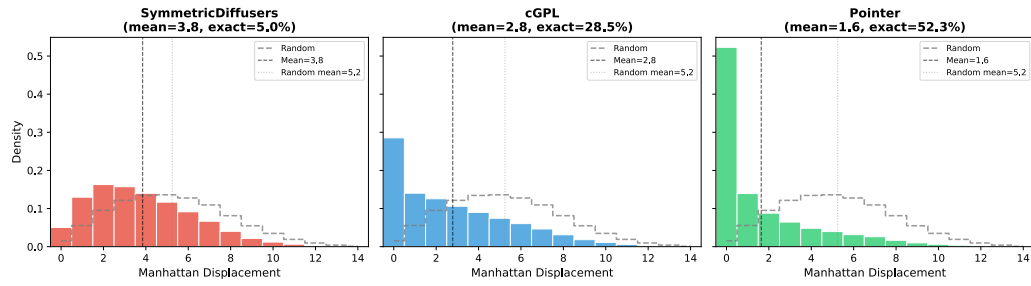


Figure 7. **Patch-wise Manhattan displacement distributions on 8×8 CIFAR-10 jigsaw.** Manhattan displacement is the grid distance between a patch’s predicted and correct positions. Dashed gray = random baseline (mean 5.2). Left: SymmetricDiffusers (mean 3.8). Middle: cGPL (mean 2.8). Right: Pointer-cGPL (mean 1.6).